

Projet RabbitMQ

Dali Zaina

Zazari Iman

2024/2025

Introduction

L'Institut de Physique Nucléaire NGI souhaite mettre en place un **système de calcul distribué** afin d'effectuer des opérations mathématiques complexes. Dans cette optique, une consultation est organisée auprès de plusieurs fournisseurs pour évaluer leur efficacité et leurs compétences à travers la réalisation d'un prototype.

Le projet consiste à développer un **système de calcul distribué simple** reposant sur **RabbitMQ**. Le fonctionnement initial sera le suivant :

- Un **client** envoie, toutes les 5 secondes, un message contenant deux nombres aléatoires (n_1 et n_2).
- Un **worker** reçoit le message, effectue l'**addition** des deux nombres, attend aléatoirement entre 5 et 15 secondes pour simuler un calcul complexe, puis renvoie le résultat.
- Un **autre client** lit la file de résultats et affiche les opérations effectuées.

Ce système de base sera progressivement enrichi par plusieurs **améliorations fonctionnelles**, afin de rendre la solution plus robuste, plus performante et mieux adaptée aux besoins futurs.

Présentation générale du projet

Ce projet a été réalisé dans le cadre d'une simulation de système de calcul distribué pour l'Institut de Physique Nucléaire NGL.

Nous avons utilisé **Python** comme langage principal pour développer les différentes parties du projet (clients, workers) et RabbitMQ comme système de messagerie.

Le code source est disponible sur notre dépôt Git public :

 https://github.com/ZainaDali/Projet_rabitMQ.git

Installation et mise en place

◆ Prérequis

Avant de commencer, assurez-vous d'avoir installé sur votre machine :

- Python 3.8+
- Git

◆ Cloner le projet

Ouvrez un terminal et tapez :

- `git clone https://github.com/ZainaDali/Projet_rabitMQ.git`
- `cd Projet_rabitMQ`

◆ Installer les dépendances Python

- `pip install pika`

Lancement du système

Composants

Le système repose sur 3 composants :

`client_send.py`

Génère des calculs (add, sub, mul, div, all) et les envoie à RabbitMQ

`worker.py`

Traite un seul type d'opération (add, sub, mul ou div), après paramétrage

`client_receive.py`

Affiche les résultats des opérations traitées

Lancer le projet

Dans 3 terminaux minimum :

1. Lancer les workers (1 terminal par worker)

```
python worker.py add
python worker.py sub
python worker.py mul
python worker.py div
```

2. Lancer le client émetteur (génère un calcul toutes les 2 à 5 secondes)

```
python client_send.py
```

3. Lancer le client récepteur de résultats

```
python client_receive.py
```

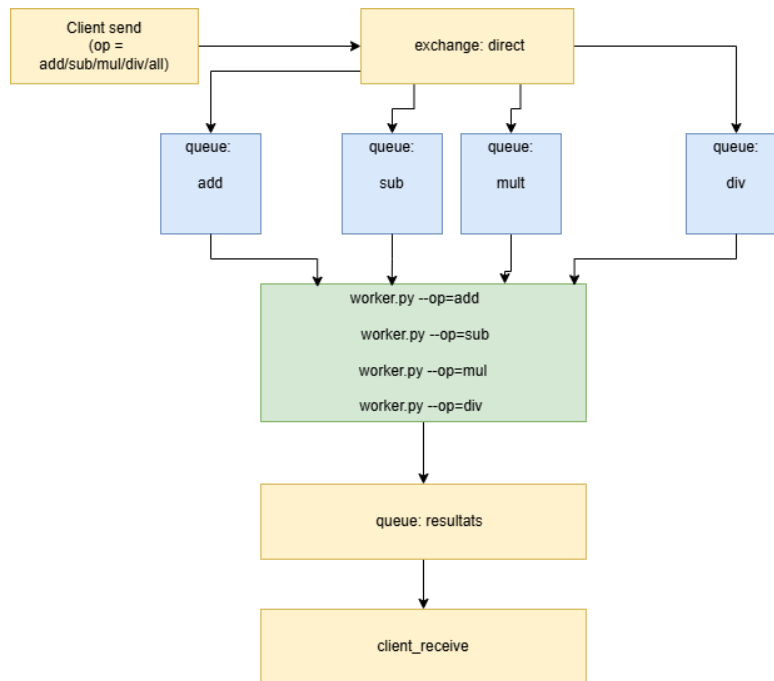
Fonctionnement du système amélioré

Le système utilise maintenant UN **exchange de type direct**, appelé `calculs_exchange`, pour répartir les requêtes selon l'opération.

Chaque `worker.py` est lancé avec une opération spécifique (add, sub, mul, div) et ne consomme **que les messages liés à cette opération**.

Un cas particulier est prévu pour l'opération `all` :

- Le client envoie un message avec `op: "all"`
- Ce message est publié **vers les 4 routing keys** (`add`, `sub`, `mul`, `div`)
- Tous les workers traitent la même paire de nombres



Interface graphique

Pour aller au-delà du fonctionnement en ligne de commande, une **interface graphique** a été développée afin de centraliser :

- L'envoi des calculs (`client_send.py`)
- Le lancement et l'arrêt des différents workers (`add`, `sub`, `mul`, `div`)
- La visualisation des résultats (`client_receive.py`)

Notre interface Tkinter permet de **lancer, suivre, et arrêter dynamiquement** plusieurs processus liés à un système de calcul, incluant un client, un récepteur de résultats, et 4 workers spécialisés (`add`, `sub`, `mul`, `div`).

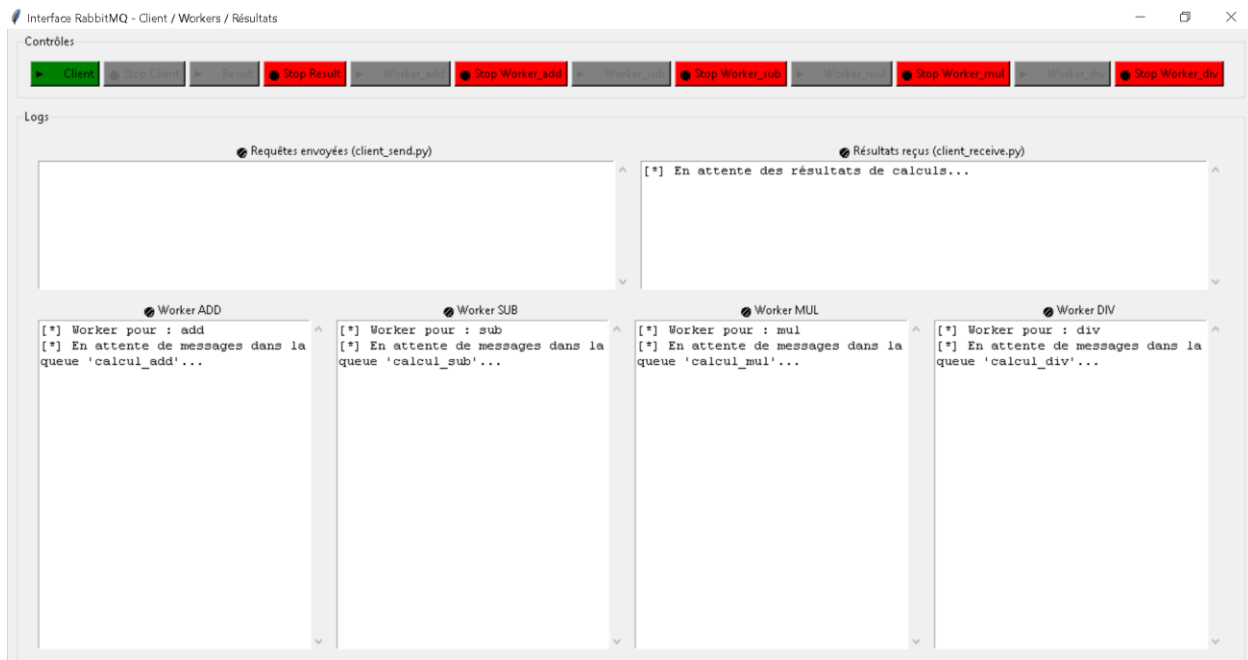
Lancement de l'interface

On peut lancer l'interface comme suit :

Python .\interface.py

1. Zone supérieure : Contrôles interactifs

- **Boutons de démarrage et d'arrêt :**
 - Chaque rôle (Client, Result, Worker ADD, SUB, MUL, DIV) dispose de **boutons colorés** :
 -  Vert = prêt à être lancé
 -  Rouge = en cours d'exécution
 -  Gris = inactif ou arrêté



2. Zone centrale : Logs et retours en temps réel

a. Requêtes envoyées (client_send.py)

- Affiche les messages envoyés par le client, incluant :
 - Les nombres n1, n2
 - L'opération choisie (op)
 - L'identifiant de requête

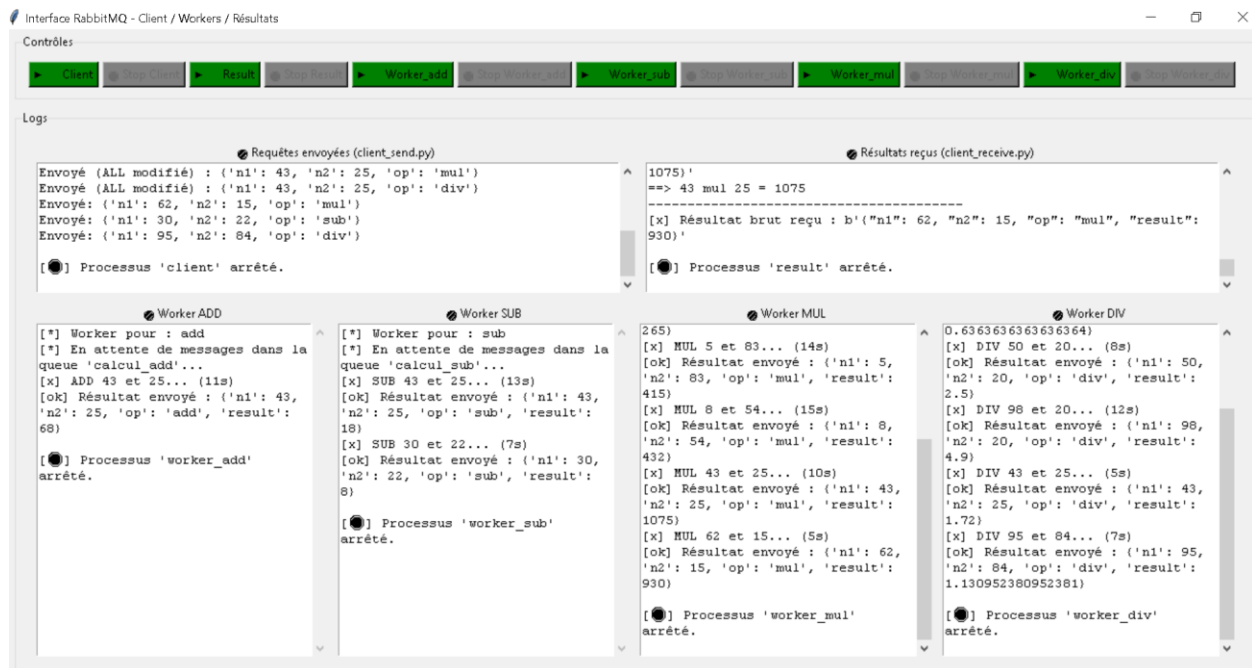
b. Résultats reçus (client_receive.py)

- Affiche les réponses calculées retournées par les workers.
- Détaille :
 - Les données brutes reçues
 - Les formules et résultats formatés (ex: $64 * 94 = 6016$)



3. Zone inférieure : Logs des workers par opération

- Chaque worker (add, sub, mul, div) est affiché dans une zone dédiée avec ses propres logs :
 - Début de traitement avec les entrées
 - Durée simulée du calcul (11s, 14s, etc.)
 - Résultat produit



Conclusion

Ce projet nous a permis de concevoir et de mettre en œuvre un système de calcul distribué robuste et évolutif à l'aide de RabbitMQ et Python.

L'ajout d'une interface graphique intuitive, développée avec Tkinter, a fortement amélioré l'expérience utilisateur en permettant de piloter et visualiser l'ensemble des composants en temps réel. Cette interface dynamique gère non seulement le lancement et l'arrêt des processus, mais offre également une visibilité claire sur les requêtes envoyées, les opérations effectuées, et les résultats obtenus.

Ce projet nous a ainsi permis de maîtriser RabbitMQ dans un contexte concret, et d'acquérir une expérience précieuse.